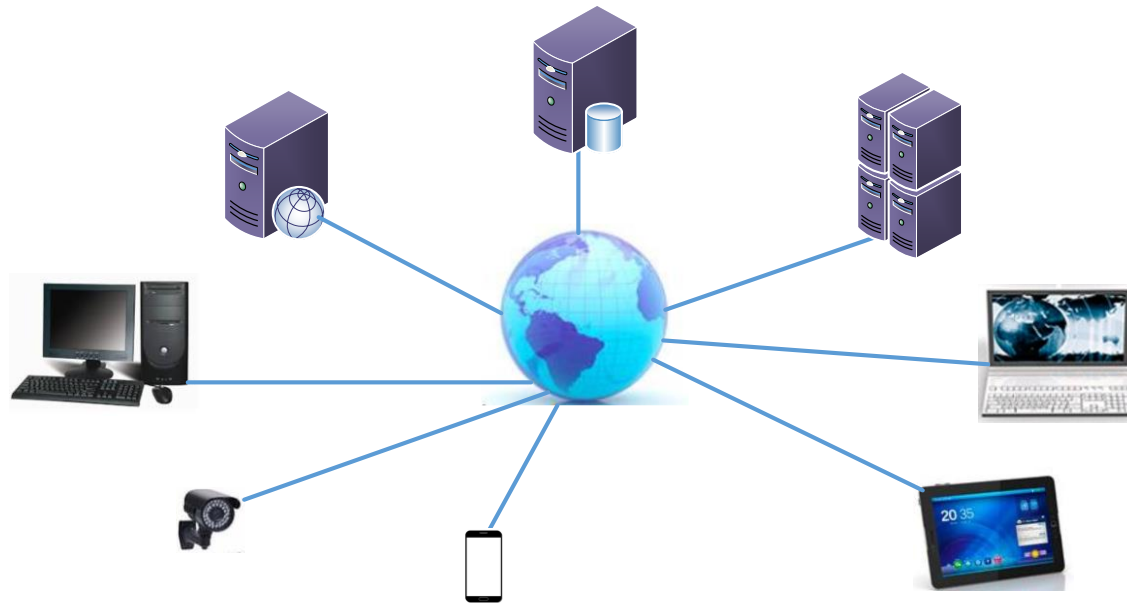


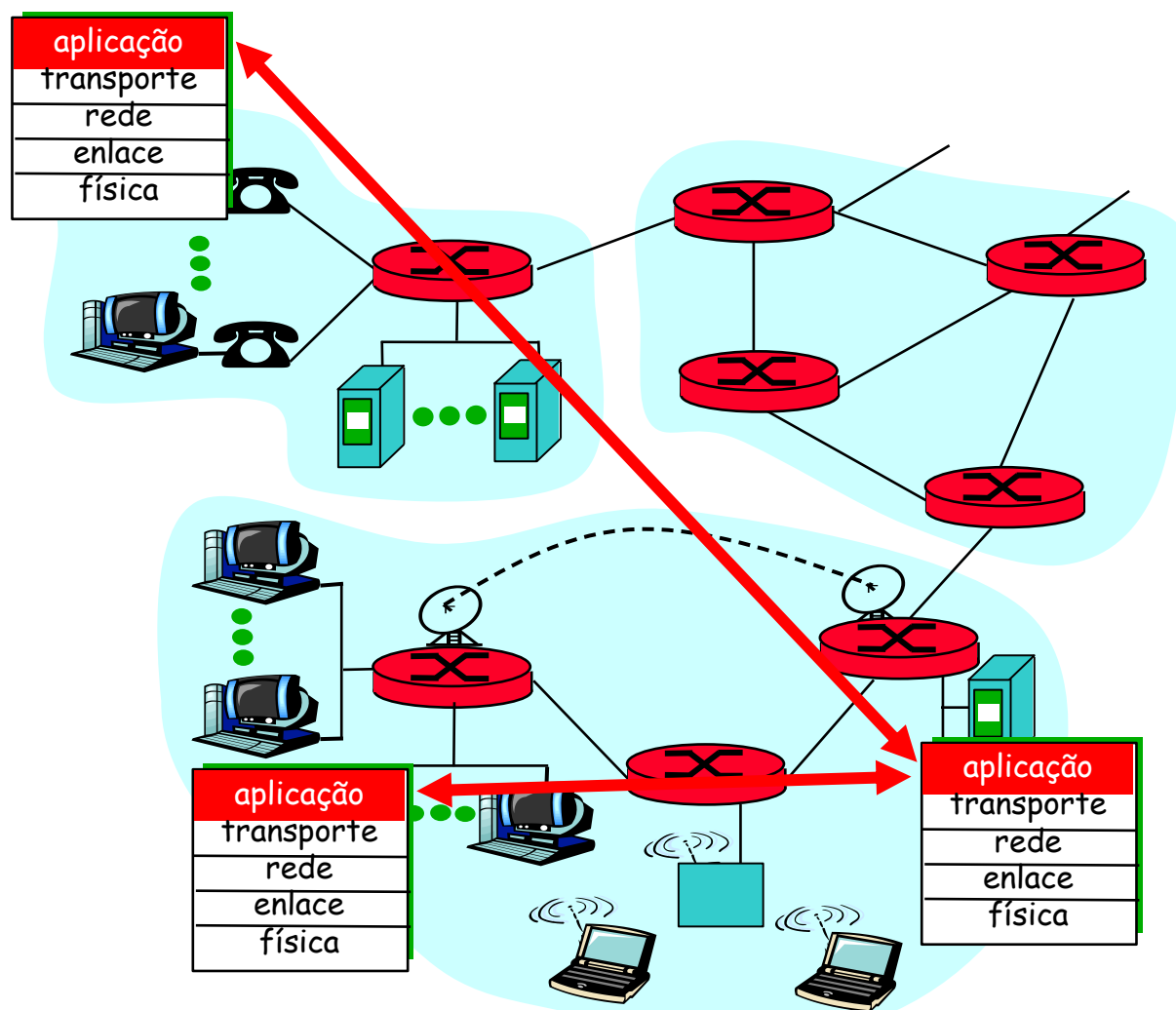
Arquitetura Cliente/Servidor



Prof. Dr. Márcio Andrey Teixeira
Instituto Federal de São Paulo – Campus Catanduva
Catanduva, SP
Membro Sênior do IEEE
marcio.andrey@ifsp.edu.br

Introdução a arquitetura cliente/servidor

Um protocolo de aplicação de rede tem normalmente duas partes ou “lados”: um **lado cliente** e um **lado servidor**. Veja a Figura a seguir:



Introdução a arquitetura cliente/servidor

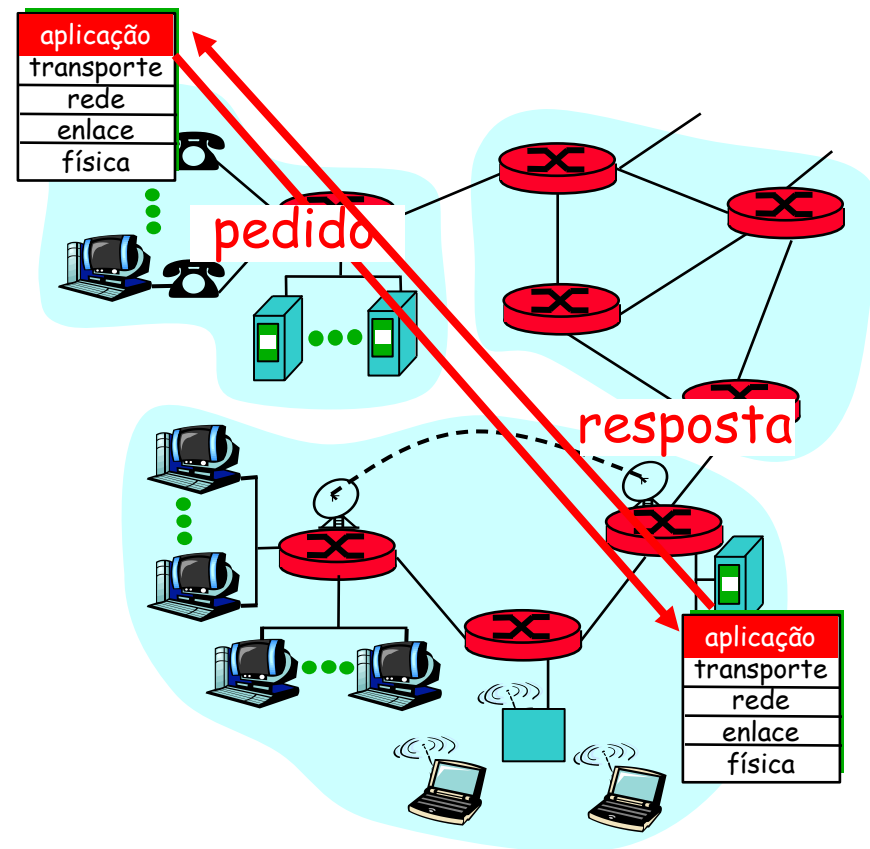
- Aplicações de rede típicas têm duas partes: *cliente* and *servidor*

- **Cliente:**

- inicia comunicação com o servidor (“fala primeiro”)
- tipicamente solicita serviços do servidor,
- Web: cliente implementado no browser; e-mail: leitor de correio

- **Servidor:**

- fornece os serviços solicitados ao cliente
- e.x., Web server envia a página Web solicitada, servidor de e-mail envia as mensagens, etc.



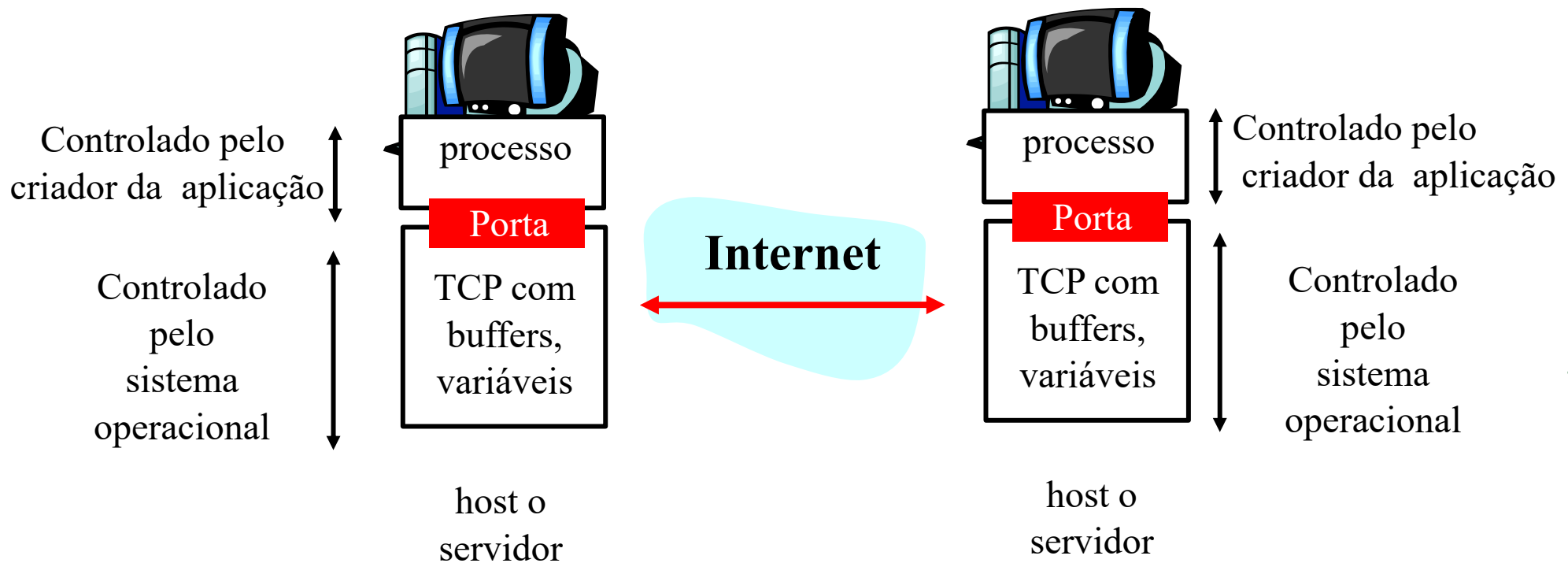
Aplicações e Protocolo de Aplicação

- **Aplicação: processos distribuídos em comunicação**
 - rodam nos computadores usuários da rede como programas de usuário
 - trocam mensagens para realização da aplicação
 - e.x., email, ftp, Web
-
- **Protocolos de aplicação**
 - fazem parte das aplicações
 - definem mensagens trocadas e as ações tomadas
 - usam serviços de comunicação das camadas inferiores

Aplicações de Rede

- **Processo:** programa executando num host.
 - dentro do mesmo host: **interprocess communication** (definido pelo OS).
 - processos executando em diferentes hosts se comunicam com um **protocolo da camada de aplicação**
- **Agente usuário:** software que interfaceia com o usuário de um lado e com a rede de outro.
 - implementa protocolo da camada de aplicação
 - Web: browser
 - E-mail: leitor de correio
 - streaming audio/video: media player

Aplicações de Rede



Como pode ser observado, a porta é uma interface entre a camada de aplicação e a camada de transporte dentro de uma máquina.

É descrita como uma *API (Application Program Interface)* entre a aplicação e a rede, uma vez que a porta é a interface de programação pela qual as aplicações de rede são inseridas na internet.

Aplicações de Rede

O desenvolvedor da aplicação tem controle de tudo o que existe no lado da camada da aplicação, mas pouco controle do lado da camada de transporte, tais como, um tamanho máximo de buffer, e um tamanho máximo de segmentos.

Uma vez que o desenvolvedor da aplicação tenha escolhido um protocolo de transporte, a aplicação é construída usando os serviços da camada de transporte oferecidos por este protocolo.

Interfaces de Programação

- **API: application programming interface**
 - Define a interface entre a camada de aplicação e de transporte
 - socket: Internet API
 - dois processos se comunicam enviando dados para o socket e lendo dados de dentro do socket
- Como um processo “identifica” o outro processo com o qual ele quer se comunicar?
 - **IP address** do computador no qual o processo remoto executa;
 - “**port number**” - permite ao computador receptor determinar o processo local para o qual a mensagem deve ser entregue.

Exemplo: `import os`

`FILE_PATH = "mensagens.txt"`

`def escrever_mensagem():`

`mensagem = input("Digite a mensagem para salvar: ")`

`with open(FILE_PATH, "a") as file:`

`file.write(mensagem + "\n")`

`print(" Mensagem salva com sucesso!\n")`

`def listar_mensagens():`

`if not os.path.exists(FILE_PATH):`

`print("Nenhuma mensagem encontrada.\n")`

`return`

`with open(FILE_PATH, "r") as file:`

`mensagens = file.readlines()`

`if mensagens:`

`print("\n Mensagens Salvas:")`

`for i, msg in enumerate(mensagens, 1):`

`print(f"{i}. {msg.strip()}")`

`else:`

`print(" Nenhuma mensagem encontrada.")`

`print()`

Exemplo:

```
def deletar_mensagem():  
    if not os.path.exists(FILE_PATH):  
        print("Nenhuma mensagem encontrada.\n")  
        return  
  
    listar_mensagens()  
    mensagens = open(FILE_PATH).readlines()  
  
    try:  
        escolha = int(input("Digite o número da mensagem para  
deletar: ")) - 1  
        if 0 <= escolha < len(mensagens):  
            mensagens.pop(escolha)  
            with open(FILE_PATH, "w") as file:  
                file.writelines(mensagens)  
            print("Mensagem deletada com sucesso!\n")  
        else:  
            print("Número inválido.\n")  
    except ValueError:  
        print("Entrada inválida. Digite um número.\n")
```

Exemplo:

```
def menu():
    while True:
        print("=== Aplicação Local - Gerenciamento de Mensagens ===")
        print("1. Escrever mensagem")
        print("2. Listar mensagens")
        print("3. Deletar mensagem")
        print("4. Sair")
        escolha = input("Escolha uma opção: ")

        if escolha == "1":
            escrever_mensagem()
        elif escolha == "2":
            listar_mensagens()
        elif escolha == "3":
            deletar_mensagem()
        elif escolha == "4":
            print("👋 Saindo...")
            break
        else:
            print(" Opção inválida!\n")

if __name__ == "__main__":
    menu()
```

Exemplo:

- `python local_app.py`

```
=== Aplicação Local - Gerenciamento de Mensagens ===  
1. Escrever mensagem  
2. Listar mensagens  
3. Deletar mensagem  
4. Sair  
Escolha uma opção:
```

Exemplo de outras APIs

- Além da **API de Sockets**, existem várias outras interfaces de comunicação que podem ser utilizadas dependendo do contexto e da necessidade da aplicação. Aqui estão algumas das mais comuns:

1 HTTP/HTTPS (REST APIs)

- **Descrição:** Comunicação baseada em **requisições HTTP**, amplamente usada para comunicação entre sistemas web e serviços distribuídos.
- **Uso:** APIs RESTful para comunicação cliente-servidor.
- **Linguagens:** Python (**Flask**, **FastAPI**), JavaScript (**Express.js**), Java (**Spring Boot**), etc.
- **Exemplo:**
 - **Cliente:** Faz uma requisição **GET** para um servidor.
 - **Servidor:** Responde com um JSON contendo os dados solicitados.

Exemplo de outras APIs

2 WebSockets

- **Descrição:** Permite comunicação bidirecional e em tempo real entre cliente e servidor, diferente do HTTP tradicional que é baseado em requisições-respostas.
- **Uso:** Chats, jogos online, notificações em tempo real.
- **Linguagens:** Python (**WebSocket**, **Django Channels**), JavaScript (**Socket.io**).
- **Exemplo:** Chat online onde clientes e servidor trocam mensagens instantaneamente.

Exemplo de outras APIs

3 RPC (Remote Procedure Call)

- **Descrição:** Permite que um programa execute funções ou métodos em outro sistema como se fossem chamadas locais.
- **Uso:** Comunicação entre microserviços e sistemas distribuídos.
- **Tecnologias:**
 - **gRPC** (Google) – Alta performance, usa **Protocol Buffers** (protobuf).
 - **XML-RPC** – Baseado em XML (mais antigo).
 - **JSON-RPC** – Baseado em JSON.
- **Exemplo:** Um cliente pode chamar `getUserData(id=123)` e receber os dados do usuário no servidor.

Exemplo de outras APIs

4 MQTT (Message Queuing Telemetry Transport)

- **Descrição:** Protocolo de comunicação **leve e eficiente**, muito utilizado em IoT (Internet das Coisas).
- **Uso:** Sensores, dispositivos IoT, automação industrial.
- **Linguagens:** Python (`paho-mqtt`), Node.js, C.
- **Exemplo:** Um sensor envia dados de temperatura para um broker MQTT, e dispositivos interessados (subscribers) recebem a atualização.

Exemplo de outras APIs

Resumo Comparativo

Interface	Comunicação	Uso Comum
Sockets	Cliente-Servidor via TCP/UDP	Redes, jogos, sistemas distribuídos
HTTP (REST API)	Baseado em requisições HTTP	Aplicações Web e Microserviços
WebSockets	Bidirecional e em tempo real	Chats, Notificações, Jogos Online
gRPC (RPC)	Chamada de funções remotas	Microserviços e Alto Desempenho
MQTT	Publicador/Assinante	IoT e Automação

Exercício

- Utilizando uma API de Socket, faça um programa cliente e um programa servidor baseando-se no programa `local_app.py` que foi mostrado em aula.
- O lado servidor irá receber as mensagens enviadas da aplicação cliente e irá imprimir tais mensagens na tela do servidor.
- O lado cliente irá se conectar com o lado servidor e enviará mensagens de texto para o servidor.



Prof. Dr. Marcio Andrey Teixeira

marcio.andrey@ifsp.edu.br

<http://marcioteixeira.pro.br>

Bibliografia

SILVA, G. M.. Segurança em sistemas Linux. 1. ed. Rio de Janeiro: Ciência Moderna, 2008. 240p.

THOMPSON, M. A.. Windows Server 2012: fundamentos. 1. ed. São Paulo: Érica, 2012. 232p.

VIANA, E. R. C.. Virtualização de servidores Linux para redes corporativas: guia prático. 1.

ed. Rio de Janeiro: Ciência Moderna, 2008. 342p.

6 - BIBLIOGRAFIA COMPLEMENTAR:

KUROSE, J. F.; ROSS, K. W.. Computer networking: a top-down approach. 6. ed. AddisonWesley, 2012. 864p.

SCHRODER, C.. Redes Linux: livro de receitas. 1. ed. Rio de Janeiro: Alta Books, 2006. 569p.